

Java Full Stack Development

2. Kafka Spring Boot



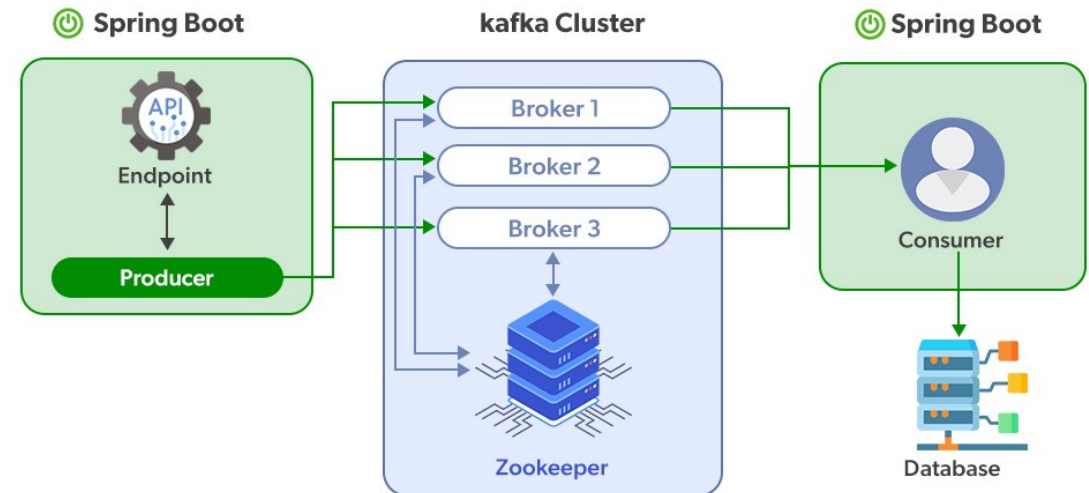
Module Topics

- The Kafka Boot Starter
- Producers
- Consumers
- Error Handling
- Transactions

Kafka Spring Boot Starter

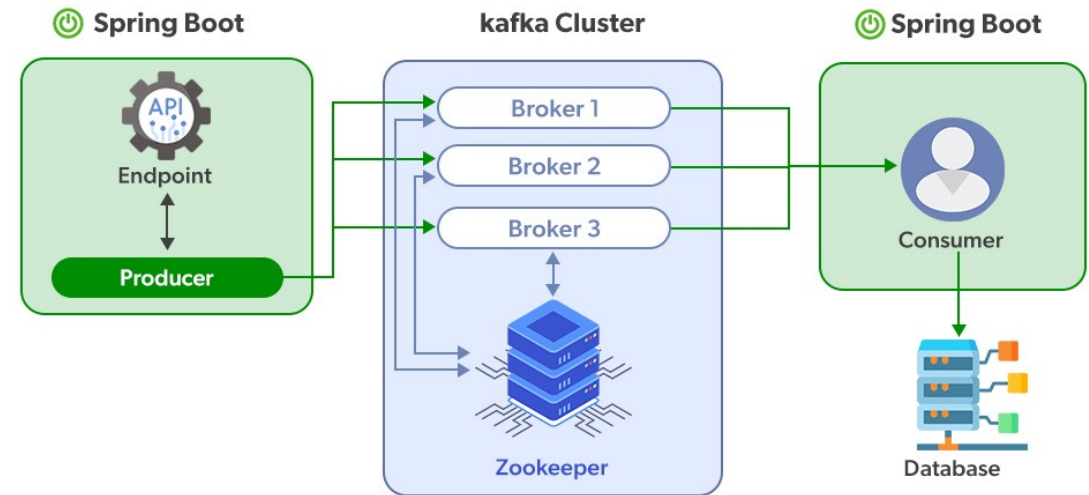
Kafka Spring Boot

- Spring Kafka provides:
 - Auto-configuration through Spring Boot
 - Declarative configuration via application.yml
 - A template abstraction for sending ([KafkaTemplate](#))
 - An annotation-driven listener model ([@KafkaListener](#))
 - Lifecycle management of consumer threads
 - Error handling, retry, and recovery infrastructure
 - Integration with Spring's transaction management



Kafka Spring Boot

- Wraps the standard Apache Kafka client
 - Doesn't replace the client
 - Adds idiomatic Spring patterns on top.
 - Kafka client requires you
 - To manage producer instances
 - Write your own poll loop in a thread
 - Handle threading and shutdown manually
 - Build your own retry logic
 - Spring Kafka provides this functionality
 - Most enterprise applications use Spring Kafka rather than the bare client



Kafka Spring Boot

- The Maven dependency
 - Spring Boot's parent POM manages the version
 - Spring Kafka provides
 - Apache Kafka clients (transitively)
 - Spring Kafka abstractions
 - Spring Boot auto-configuration support
 - Auto-configuration activates when the dependency is on the classpath

```
<dependency>  
  <groupId>org.springframework.kafka</groupId>  
  <artifactId>spring-kafka</artifactId>  
</dependency>
```

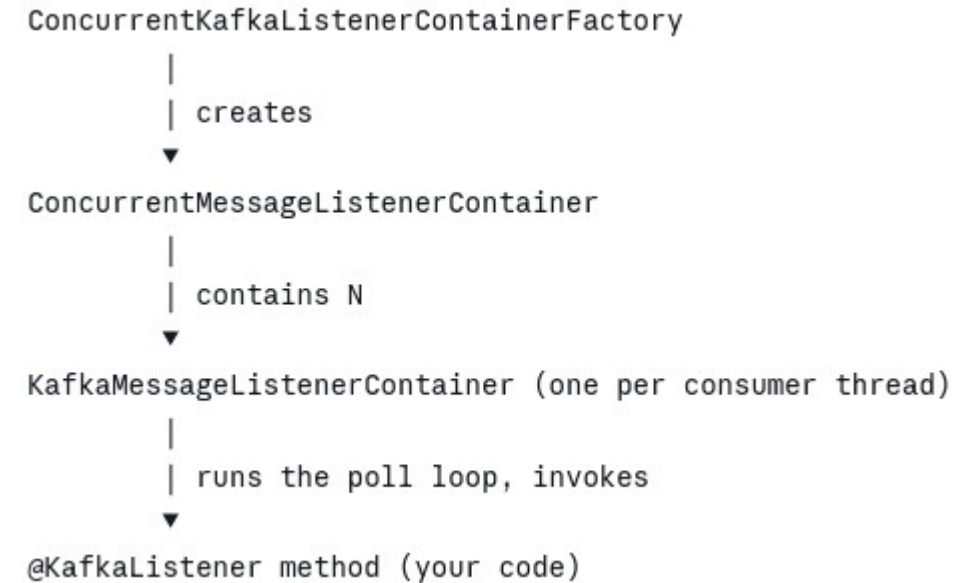
Kafka Spring Boot

- Has a layered design
- From the top
 - [KafkaTemplate](#) for sending and [@KafkaListener](#) for receiving
 - What application developers work with daily
 - Underneath, are factories
 - Produce client instances configured with the properties you set
 - [ProducerFactory](#) and [ConsumerFactory](#).
 - Below that is the container infrastructure
 - Runs the actual poll loop and invokes your listener methods.

Abstraction	Purpose
KafkaTemplate<K, V>	Send messages — wraps a KafkaProducer
ProducerFactory<K, V>	Creates KafkaProducer instances; injected into the template
@KafkaListener	Method annotation to receive messages
ConsumerFactory<K, V>	Creates KafkaConsumer instances
ConcurrentKafkaListenerContainerFactory	Builds containers that run listener methods
MessageListenerContainer	Runs the poll loop and dispatches to listeners

Kafka Spring Boot

- The container hierarchy
 - For consumers
 - The Concurrent container splits work across multiple consumer threads
 - Each `KafkaMessageListenerContainer` runs one consumer with its own poll loop
 - Concurrency level controls how many consumers run within one process



Producers

Configuration

- Config for producers
 - Bootstrap-servers
 - List of brokers to connect to initially, usually two or three for redundancy in production.
 - The serializers
 - How keys and values are converted to bytes
 - StringSerializer for the key is the most common choice when keys are entity IDs as strings
 - JsonSerializer, Avro and Protobuf serializers are common choices
 - acks: all and enable.idempotence
 - Together give durable, no-duplicates configuration we

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      retries: 10
      properties:
        enable.idempotence: true
        max.in.flight.requests.per.connection: 5
        linger.ms: 10
        compression.type: lz4
```

Configuration

- Config for producers
 - retries: 10 is generous;
 - With idempotence enabled, can retry aggressively without creating duplicates
 - linger.ms: 10
 - Enables small-batch optimization.
 - compression.type: lz4
 - Is a good general-purpose choice

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      retries: 10
      properties:
        enable.idempotence: true
        max.in.flight.requests.per.connection: 5
        linger.ms: 10
        compression.type: lz4
```

Sending a Message

- KafkaTemplate
 - Auto-configured and injectable
 - send(topic, key, value) is the most common form
 - The send is asynchronous
 - Returns a [CompletableFuture](#)
 - For fire-and-forget cases, that's all you need

```
@Service
public class OrderEventPublisher {

    private final KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate;

    public OrderEventPublisher(KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publish(OrderPlacedEvent event) {
        kafkaTemplate.send("orders.placed", event.orderId(), event);
    }
}
```

Sending a Message

- Most basic producer code
 - For many use cases it's all that is needed
 - Inject `KafkaTemplate`, call `send()`
 - The template is generic
 - `KafkaTemplate<String, OrderPlacedEvent>`
 - The type parameters must align with the configured serializers
 - Key type is `String` because we configured `StringSerializer`
 - Value type is `OrderPlacedEvent` because `JsonSerializer` can handle any serializable object.

```
@Service
public class OrderEventPublisher {

    private final KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate;

    public OrderEventPublisher(KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publish(OrderPlacedEvent event) {
        kafkaTemplate.send("orders.placed", event.orderId(), event);
    }
}
```

Sending a Message

- Most basic producer code
 - The send() call returns a [CompletableFuture](#)
 - Can be ignored for fire-and-forget cases
 - Or attach a callback to for delivery confirmation.
 - Behind the scenes
 - Spring routed the call through the [ProducerFactory](#) to a real [KafkaProducer](#)
 - Which serialized the event, decided on a partition based on the key, and queued it for sending
 - The application thread is unblocked immediately

```
@Service
public class OrderEventPublisher {

    private final KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate;

    public OrderEventPublisher(KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publish(OrderPlacedEvent event) {
        kafkaTemplate.send("orders.placed", event.orderId(), event);
    }
}
```

The Send Result

- The future completes
 - When the broker acknowledges or the send fails
 - Success: you get the partition and offset assigned
 - Failure: you get the exception that caused the failure
 - Don't block on the future
 - It defeats the async pipeline

```
public void publish(OrderPlacedEvent event) {
    CompletableFuture<SendResult<String, OrderPlacedEvent>> future =
        kafkaTemplate.send("orders.placed", event.orderId(), event);

    future.whenComplete((result, ex) -> {
        if (ex != null) {
            log.error("Failed to publish event for order {}", event.orderId(),
                ex);
        } else {
            RecordMetadata metadata = result.getRecordMetadata();
            log.debug("Published to {}-{} @ offset {}",
                metadata.topic(), metadata.partition(), metadata.offset());
        }
    });
}
```


The Send Result

- For most production code
 - You'll want to know whether the send succeeded
 - At minimum to log failures so you can detect them
 - The `whenComplete` callback is the idiomatic way to handle this
 - It runs when the future completes, with either the result or an exception
 - On success, you get a `SendResult` containing `RecordMetadata` with the partition and offset that were assigned
 - On failure, you get the exception, typically a network issue, a broker rejection, or a serialization problem

```
public void publish(OrderPlacedEvent event) {
    CompletableFuture<SendResult<String, OrderPlacedEvent>> future =
        kafkaTemplate.send("orders.placed", event.orderId(), event);

    future.whenComplete((result, ex) -> {
        if (ex != null) {
            log.error("Failed to publish event for order {}", event.orderId(),
                ex);
        } else {
            RecordMetadata metadata = result.getRecordMetadata();
            log.debug("Published to {}-{} @ offset {}",
                metadata.topic(), metadata.partition(), metadata.offset());
        }
    });
}
```

The Send Result

- Headers carry metadata separate from the payload
 - Common uses
 - Correlation IDs, tracing context, schema versions, source identifiers
 - Headers travel with the record through Kafka and are available to consumers
 - Don't put domain data in headers
 - They're for cross-cutting concerns

```
public void publish(OrderPlacedEvent event, String correlationId) {  
    var record = MessageBuilder  
        .withPayload(event)  
        .setHeader(KafkaHeaders.TOPIC, "orders.placed")  
        .setHeader(KafkaHeaders.KEY, event.orderId())  
        .setHeader("correlationId", correlationId)  
        .setHeader("source", "order-service")  
        .build();  
  
    kafkaTemplate.send(record);  
}
```

JSON Serialization

- Uses Jackson under the hood
 - Serializes the value object to a JSON byte array
 - By default, includes a type header (`__TypeId__`) so consumers can deserialize
 - Works for any class Jackson can serialize
 - Configurable via `JsonSerializer` properties

```
spring:
  kafka:
    producer:
      properties:
        spring.json.add.type.headers: false # disable type
                                           # info if consumer is non-Spring
```

Consumers

Consumer Properties

- This is a complete, production-leaning consumer configuration
 - The group-id identifies the consumer group
 - This is part of the service's identity and should be stable across deployments
 - The deserializers mirror the serializers on the producer side
 - The producer used JsonSerializer, the consumer uses JsonDeserializer
 - auto-offset-reset: earliest
 - Controls what happens when there's no committed offset for this group
 - Typically because it's a brand new group
 - Starts at the beginning of the topic.

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: order-inventory-updater
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      auto-offset-reset: earliest
      enable-auto-commit: false
      max-poll-records: 100
      properties:
        spring.json.trusted.packages: "com.example.events"
        max.poll.interval.ms: 300000
        session.timeout.ms: 45000
```

Consumer Properties

- This is a complete, production-leaning consumer configuration
 - enable-auto-commit: false
 - It tells Spring Kafka that we'll handle offset commits explicitly, which gives us more control over delivery semantics
 - max-poll-records
 - Limits how many records each poll() call returns

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: order-inventory-updater
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      auto-offset-reset: earliest
      enable-auto-commit: false
      max-poll-records: 100
    properties:
      spring.json.trusted.packages: "com.example.events"
      max.poll.interval.ms: 300000
      session.timeout.ms: 45000
```

Basic Listener

- The annotation registers the method as a Kafka listener
 - Spring Kafka builds a container, joins the consumer group, and dispatches records to this method
 - The method parameter type drives deserialization via the configured JsonSerializer
 - Threading, polling, and group coordination are all handled by the container

```
@Service
public class OrderEventListener {

    @KafkaListener(topics = "orders.placed")
    public void handleOrderPlaced(OrderPlacedEvent event) {
        log.info("Received order placed event: {}", event.orderId());
        // process the event
    }
}
```

Basic Listener

- Minimal consumer code
 - `@KafkaListener` annotation tells Spring Kafka to set up a consumer that calls this method for each received event
 - By default, the consumer joins the group specified in `application.yml`, then
 - Polls the configured topics
 - Deserializes records using the configured deserializer
 - Invokes the method with the deserialized value.
 - The poll loop, threading, heartbeats, group coordination, error recovery
 - Are handled by the container that Spring Kafka built around your method
 - The code looks like an ordinary handler

```
@Service
public class OrderEventListener {

    @KafkaListener(topics = "orders.placed")
    public void handleOrderPlaced(OrderPlacedEvent event) {
        log.info("Received order placed event: {}", event.orderId());
        // process the event
    }
}
```


Method Signatures

- Listener model is flexible about method signatures
 - The choice depends on what you need
 - Just the value
 - Simplest and works when you only care about the payload.
 - Value plus key
 - Adding `@Header` parameters lets you pull specific headers
 - Or known Kafka metadata like the key, partition, offset, timestamp into typed parameters

```
// Just the value
@KafkaListener(topics = "orders.placed")
public void handle(OrderPlacedEvent event) { ... }

// Value plus key
@KafkaListener(topics = "orders.placed")
public void handle(@Payload OrderPlacedEvent event,
                  @Header(KafkaHeaders.RECEIVED_KEY) String key) { ... }

// Full record access
@KafkaListener(topics = "orders.placed")
public void handle(ConsumerRecord<String, OrderPlacedEvent> record) { ... }

// With manual acknowledgment
@KafkaListener(topics = "orders.placed")
public void handle(OrderPlacedEvent event, Acknowledgment ack) {
    process(event);
    ack.acknowledge();
}
```

Method Signatures

- Listener model is flexible about method signatures
 - Full consumer record
 - Gives you everything: key, value, headers, partition, offset, timestamp, in one object
 - Useful when you need rich access to the record.
 - Manual ack pattern
 - Adding an Acknowledgment parameter
 - Spring Kafka doesn't commit the offset until you call `ack.acknowledge()`
 - This gives you explicit control over when an offset is considered processed
 - The right choice depends on the listener's needs

```
// Just the value
@KafkaListener(topics = "orders.placed")
public void handle(OrderPlacedEvent event) { ... }

// Value plus key
@KafkaListener(topics = "orders.placed")
public void handle(@Payload OrderPlacedEvent event,
                  @Header(KafkaHeaders.RECEIVED_KEY) String key) { ... }

// Full record access
@KafkaListener(topics = "orders.placed")
public void handle(ConsumerRecord<String, OrderPlacedEvent> record) { ... }

// With manual acknowledgment
@KafkaListener(topics = "orders.placed")
public void handle(OrderPlacedEvent event, Acknowledgment ack) {
    process(event);
    ack.acknowledge();
}
```

Concurrency

- Concurrency
 - How many consumer threads in this process for this listener
 - Each thread is a separate group member with its own poll loop
 - Effective max parallelism = $\min(\text{concurrency}, \text{partition count})$
 - Use it to scale within a single instance

```
spring:  
  kafka:  
    listener:  
      concurrency: 4
```

```
@KafkaListener(topics = "orders.placed", concurrency = "4")  
public void handle(OrderPlacedEvent event) { ... }
```

Concurrency

- Setting concurrency to 4
 - Spring Kafka starts four consumer threads
 - Each with its own consumer instance
 - Each running its own poll loop
 - All sharing the same group ID
 - From Kafka's perspective
 - This looks like four group members
 - They get partitions assigned via the normal group coordination
 - Partitions are spread across the threads. The hard ceiling is the partition count
 - Example
 - If the topic has 4 partitions and you set concurrency to 8, four of your threads sit idle

```
spring:  
  kafka:  
    listener:  
      concurrency: 4
```

```
@KafkaListener(topics = "orders.placed", concurrency = "4")  
public void handle(OrderPlacedEvent event) { ... }
```

Commit Modes

- Ack modes
 - BATCH mode
 - Commits after each batch of records returned from a poll() is fully processed
 - RECORD mode
 - Commits after every single record
 - Safer but more overhead
 - TIME and COUNT
 - Variants are for scenarios where you want to commit periodically rather than per-batch
 - MANUAL
 - Gives explicit control: the container only commits when you call acknowledge()
 - The difference between MANUAL and MANUAL_IMMEDIATE is when the actual commit happens
 - At the next poll, or right away

Mode	Behavior
RECORD	Commit after each record is processed
BATCH	Commit after a batch of records is processed (default)
TIME	Commit periodically (every N ms)
COUNT	Commit after N records
COUNT_TIME	Commit after N records or N ms, whichever comes first
MANUAL	Listener calls <code>ack.acknowledge()</code> , container commits at next poll
MANUAL_IMMEDIATE	Listener calls <code>ack.acknowledge()</code> , container commits immediately

Commit Modes

- Manual ack pattern in action
 - yaml configuration sets the listener factory to manual ack mode
 - The listener method takes an Acknowledgment parameter and only calls acknowledge() on successful processing
 - If processing throws, the ack is never called, and Kafka's view of the consumer's progress doesn't advance past this record
 - On the next poll (or after a rebalance), the record will be re-delivered.
 - This gives at-least-once semantics with very explicit control
 - You decide what counts as "successfully processed"

```
spring:
  kafka:
    listener:
      ack-mode: manual
```

```
@KafkaListener(topics = "orders.placed")
public void handle(OrderPlacedEvent event, Acknowledgment ack) {
    try {
        processOrder(event);
        ack.acknowledge(); // commit only on success
    } catch (TransientException e) {
        // don't ack - record will be re-delivered
        throw e;
    }
}
```

Consumer Container

- These shape how the container runs the consumer
 - Most defaults are reasonable for typical workloads
 - Worth knowing what's available when something needs adjusting
- The listener container has its own set of tune-able parameters
 - Separate from the underlying Kafka consumer properties
 - These control how the container itself behaves around the consumer

```
spring:
  kafka:
    listener:
      type: single           # single or batch listener
      ack-mode: batch        # see previous slides
      concurrency: 3         # threads per listener
      poll-timeout: 5000     # max wait for poll() in ms
      missing-topics-fatal: false # fail at startup if topic doesn't exist?
      idle-between-polls: 0  # delay between polls (ms)
```

Error Handling

Error Handling

- DefaultErrorHandler
 - Accepts a BackOff strategy
 - **FixedBackOff** waits a constant time between retries
 - **ExponentialBackOff** doubles the wait each time up to a cap
 - The retry count is bounded, you don't retry forever
 - After the max attempts, the handler calls its configured recoverer
 - While a record is being retried, the container is paused on that partition

```
@Configuration
public class KafkaErrorHandlingConfig {

    @Bean
    public DefaultErrorHandler errorHandler() {
        FixedBackOff backOff = new FixedBackOff(1000L, 3); // 1s between, 3 attempts
        return new DefaultErrorHandler(backOff);
    }
}
```

```
ExponentialBackOffWithMaxRetries backOff = new ExponentialBackOffWithMaxRetries(5);
backOff.setInitialInterval(500L);
backOff.setMultiplier(2.0);
backOff.setMaxInterval(10_000L);
return new DefaultErrorHandler(backOff);
```

Error Handling

- Some exceptions indicate a problem retry won't fix
 - Examples
 - Invalid event format
 - Validation failure
 - Programming error
 - Configure them as non-retryable
 - They go straight to the recoverer
 - Saves time and reduces noise in logs

```
DefaultExceptionHandler handler = new DefaultExceptionHandler(backOff);

// Don't retry these – fail fast and recover
handler.addNotRetryableExceptions(
    ValidationException.class,
    DeserializationException.class,
    IllegalArgumentException.class
);

return handler;
```

Dead-Letter Topics

- Where bad records go
 - After retries are exhausted or for non-retryable exceptions
 - The record is published to a dead-letter topic
 - DLT name convention: <original-topic>.DLT
 - The DLT preserves the failed record plus headers describing the failure
 - A separate process can inspect, repair, or replay DLT records

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(template,
        (record, ex) -> new TopicPartition(record.topic() + ".DLT", record.partition()));

    DefaultErrorHandler handler = new DefaultErrorHandler(recoverer,
        new FixedBackoff(1000L, 3));

    handler.addNotRetryableExceptions(ValidationException.class);
    return handler;
}
```

Dead-Letter Topics

- Standard way to handle records that can't be processed
 - After retries are exhausted, the `DeadLetterPublishingRecoverer` publishes the failed record to a separate "dead-letter" topic
 - Then commits the offset on the original topic so processing can continue past it
 - The headers added to the DLT record include the original topic, partition, offset, exception class, exception message, and stack trace
 - This is enough information for a human or an automated process to inspect what went wrong
 - The DLT itself is just a Kafka topic

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(template,
        (record, ex) -> new TopicPartition(record.topic() + ".DLT", record.partition()));

    DefaultErrorHandler handler = new DefaultErrorHandler(recoverer,
        new FixedBackOff(1000L, 3));

    handler.addNotRetryableExceptions(ValidationException.class);
    return handler;
}
```

Deserialization Failures

- Deserialization happens before the listener method is invoked
 - A deserialization failure normally crashes the consumer
 - `ErrorHandlingDeserializer` wraps another deserializer and converts failures into errors the error handler can route to the DLT
 - This is essential for production
 - Without it, one bad record blocks the partition

```
spring:
  kafka:
    consumer:
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.trusted.packages: "com.example.events"
```

Designing for Poison Pills

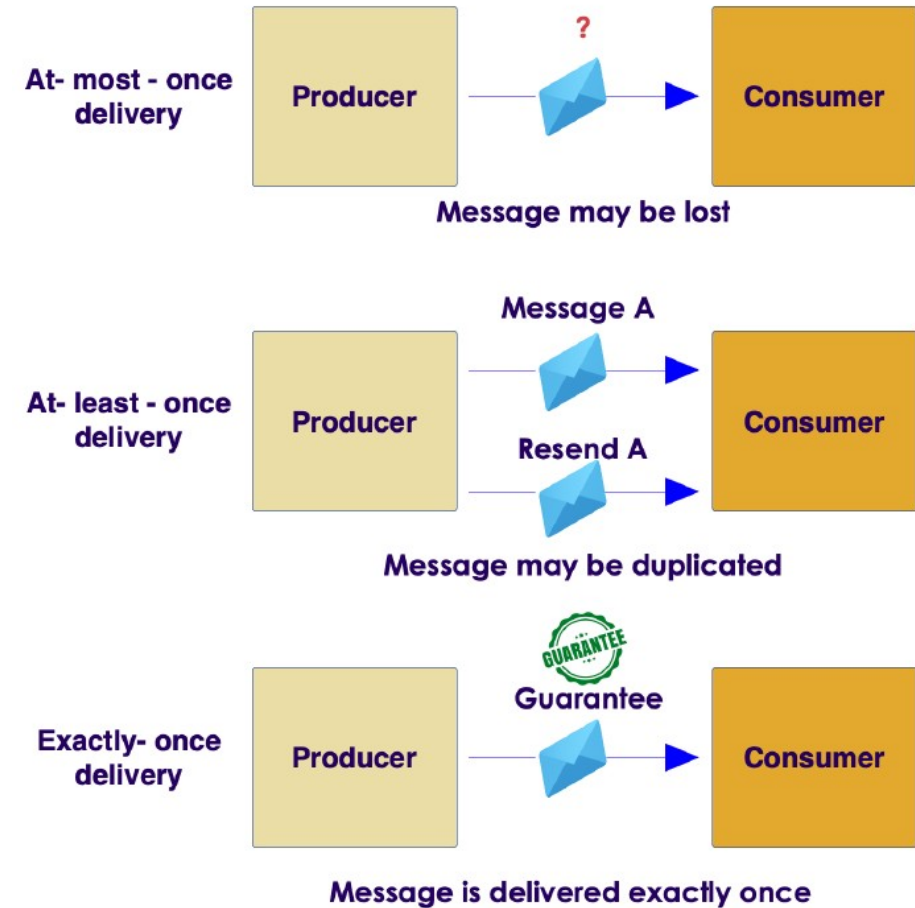
- Assume any record can fail in any way
 - Validate early, fail fast, mark as non-retryable
 - Use `ErrorHandlingDeserializer` so malformed records don't block the partition
 - Configure a dead-letter topic for every consumer
 - Monitor the DLT
 - Bad records arriving there are signals, not noise
 - Have a process for inspecting and replaying DLT records (manual or automated)

```
spring:
  kafka:
    consumer:
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
    properties:
      spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
      spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.JsonDeserializer
      spring.json.trusted.packages: "com.example.events"
```

Transactions

Exactly Once Delivery

- "Exactly-once" in Kafka has a specific meaning
 - It refers to the consume-process-produce pattern:
 - Consume a record
 - Do some processing
 - Produce a result record
 - Commit the consumer offset
 - All as one atomic unit
 - It does not mean:
 - Side effects on external systems happen exactly once
 - Records are physically delivered exactly once
 - Network and storage failures are eliminated



Configuring a Transactional Producer

- Setting transaction-id-prefix enables transactional mode
 - Each producer instance gets a unique transactional ID derived from the prefix
 - Transactions require idempotence (auto-enabled when transactions are on)
 - The prefix should be stable per logical producer for fencing to work correctly

```
spring:
  kafka:
    producer:
      transaction-id-prefix: order-service-tx-
      acks: all
      properties:
        enable.idempotence: true
```

Configuring a Transactional Producer

- Each producer instance Spring creates
 - Will get a unique transactional ID built from this prefix
 - Typically by appending a sequence number or instance identifier
 - Kafka uses it to handle "fencing"
 - If a producer crashes mid-transaction and a replacement starts up with the same transactional ID
 - The broker knows the old transaction is dead and won't accept any further writes from the old instance
 - This prevents zombie producers from corrupting state.

```
spring:
  kafka:
    producer:
      transaction-id-prefix: order-service-tx-
      acks: all
      properties:
        enable.idempotence: true
```

Sending Within a Transaction

- Spring's `@Transactional` integrates with the Kafka transaction manager
 - All sends within the method are part of one Kafka transaction
 - An exception causes a rollback
 - None of the records become consumer-visible
 - A successful return commits the transaction

```
@Service
public class OrderProcessor {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    @Transactional // Spring's transactional annotation
    public void processOrder(Order order) {
        kafkaTemplate.send("orders.validated", order.id(), validatedEvent(order));
        kafkaTemplate.send("inventory.reserve", order.id(), reserveEvent(order));
        kafkaTemplate.send("audit.log", order.id(), auditEvent(order));
        // all three sends succeed together, or none become visible to consumers
    }
}
```

Sending Within a Transaction

- With a transactional producer configured
 - You can use Spring's `@Transactional` annotation on methods that produce to Kafka
 - All the sends within that method are part of a single Kafka transaction
 - If the method throws, the transaction is rolled back
 - None of the produced records become visible to consumers
 - Assuming consumers are reading committed only, which we'll cover next)
 - If the method completes successfully
 - The transaction commits and all records become visible together

```
@Service
public class OrderProcessor {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    @Transactional // Spring's transactional annotation
    public void processOrder(Order order) {
        kafkaTemplate.send("orders.validated", order.id(), validatedEvent(order));
        kafkaTemplate.send("inventory.reserve", order.id(), reserveEvent(order));
        kafkaTemplate.send("audit.log", order.id(), auditEvent(order));
        // all three sends succeed together, or none become visible to consumers
    }
}
```

Reading Committed Records Only

- Default is read_uncommitted
 - Consumers see all records as they're written
 - With read_committed
 - Consumers only see records from committed transactions
 - Aborted transaction records are filtered out
 - Required for exactly-once consumer behavior
 - Does not affect non-transactional records which are always visible

```
spring:  
  kafka:  
    consumer:  
      isolation-level: read_committed
```

Consume-Process-Produce

- The classic pattern for stream processing
 - Without transactions
 - Risk of double-publish
 - Send succeeds
 - Offset commit fails
 - Or lost work: offset commits, send fails
 - With transactions
 - Both happen, or neither happens

```
@KafkaListener(topics = "orders.placed")
@Transactional // ties consume offset to produce in one Kafka transaction
public void process(OrderPlacedEvent event) {
    EnrichedOrder enriched = enrichmentService.enrich(event);
    kafkaTemplate.send("orders.enriched", enriched.orderId(), enriched);
    // on successful return:
    //   - the produced "enriched" record becomes consumer-visible, AND
    //   - the consumed offset is committed
    // both atomically
}
```

Limits of Kafka Transactions

- Transactions are internal to Kafka
- They do not extend to:
 - Database writes (no XA / two-phase commit support)
 - HTTP calls to external services
 - File system operations
 - Any side effect outside Kafka

Scenario	Transactional?
Publish a single event, no further processing	No
Publish multiple events that must all succeed or all fail	Yes
Stream processing: consume, transform, produce	Yes
Publish to Kafka and update a database	Outbox pattern, not transactions
Send fire-and-forget telemetry	No
Need exactly-once guarantees within Kafka	Yes
Cross-system exactly-once delivery	Use idempotency in receivers

Questions?

